```
# Python Break Statement
# The break statement in Python is used to immediately terminate a for or while loop when a speci

# Example: Let's start with an example to understand how break works in a loop.


a = [1, 3, 5, 7, 9, 11]
val = 7

for i in a:
    if i == val:
        print(f"Found at {i}!")
        break
else:
    print(f"not found")
```

```
Found at 7!
```

```
# Break Statement with for Loop
# A for loop in Python iterates over a sequence (like a list, tuple, string or range) and execute

for i in range(10):
    print(i)
    if i == 6:
        break
```

```
0
1
2
3
4
5
6
```

```
# Break Statement with while Loop
# A while loop in Python repeatedly executes a block of code as long as a specified condition is


c = 5

while True:
    print(c)
    c -= 1
    if c == 0:
        print("Countdown finished!")
        break  # Exit the loop
c = 5

while True:
    print(c)
    c -= 1
    if c == 0:
        print("Countdown finished!")
        break  # Exit the loop
```

```
5
4
3
2
1
Countdown finished!
5
4
```

```
3
2
1
Countdown finished!
```

```
# Using break in Nested Loops
# Nested loops are loops within loops, allowing for more complex iterations, such as iterating ove

# Innermost Loop Exit: A break statement will only exit the loop in which it is directly placed (t
# Exiting Multiple Loops: To exit multiple levels of nested loops, additional strategies are requi
# Example: If we have a list of lists (a 2D list) and we want to find a specific number. Once we f


matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
val = 5
found = False
for r in matrix:
    for n in r:
        if n == val:
            print(f"{val} found!")
            found = True
            break  # Exit the inner loop
    if found:
        break  # Exit the outer loop
```

```
5 found!
```